

ShopMicro — Lista de Mejoras

Plan de mejora técnica orientado a seguridad, calidad, arquitectura, DevOps y presentación profesional del proyecto.

Resumen de prioridades

Prioridad	Objetivo	Resultado esperado
1	Seguridad	Eliminar secretos por defecto y endurecer APIs
2	Testing	Cobertura de lógica y flujos críticos
3	CI/CD	Validación automática de cada cambio
4	Infraestructura	Entorno reproducible con PostgreSQL + Docker
5	Arquitectura	Consistencia de stock y órdenes
6	Observabilidad	Logs, health/readiness, métricas y trazabilidad
7	Frontend/API	Validación, errores, documentación y pruebas
8	Portfolio	README, ADRs y flujo profesional de GitHub

Fase 1 — Seguridad

1. Eliminar credenciales y secretos por defecto: PostgreSQL, JWT y ADMIN_API_KEY.
2. Hacer obligatorias las variables sensibles y detener el servicio si faltan.
3. Revisar .gitignore para evitar subir .env, secretos, caches y dependencias.
4. Restringir CORS a los orígenes permitidos.
5. Agregar rate limiting para login, registro y endpoints administrativos.
6. Mejorar JWT: secreto fuerte, expiración configurable y evaluar refresh tokens.

Fase 2 — Testing y calidad

1. Agregar pytest a los servicios Python.
2. Crear pruebas para autenticación: registro, login, JWT y /me.
3. Crear pruebas para productos: CRUD, validaciones y stock.
4. Crear pruebas para órdenes: autenticación, cálculo y errores.
5. Crear pruebas de integración: auth → order → product → reserva de stock.
6. Medir cobertura y establecer un objetivo razonable de cobertura.

Fase 3 — CI/CD

1. Crear .github/workflows/ci.yml.
2. Ejecutar en cada push/PR: checkout, instalación, lint, tests y cobertura.
3. Construir las imágenes Docker durante CI para detectar errores de build.
4. Agregar badge de CI al README.
5. Evaluar análisis de seguridad de dependencias e imágenes Docker.

Fase 4 — Base de datos y entorno reproducible

1. Agregar PostgreSQL al docker-compose.yml.

2. Agregar volumen persistente para PostgreSQL.
3. Agregar healthcheck de PostgreSQL.
4. Usar depends_on condicionado a que la base esté saludable.
5. Mantener el concepto de una base de datos por microservicio.
6. Eliminar la dependencia obligatoria de una IP externa para ejecutar el proyecto.

Fase 5 — Migraciones de base de datos

1. Reemplazar db.create_all() por Flask-Migrate/Alembic.
2. Crear migraciones versionadas.
3. Documentar cómo ejecutar y revertir migraciones.
4. Preparar el esquema para despliegues reproducibles.

Fase 6 — Consistencia de stock y órdenes

1. Agregar idempotencia al proceso de creación de órdenes.
2. Definir qué ocurre si se reserva stock pero falla la creación de la orden.
3. Implementar compensación/liberación de stock cuando corresponda.
4. Documentar la estrategia de consistencia entre order-service y product-service.
5. Evaluar arquitectura orientada a eventos solo si aporta una solución real al problema.

Fase 7 — Validación de APIs

1. Agregar validación estructurada de requests con Pydantic, Marshmallow o Flask-Smorest.
2. Validar tipos y rangos: precio, stock, cantidades y email.
3. Rechazar campos arbitrarios enviados por el cliente.
4. Validar entradas antes de acceder a la base de datos o a otros servicios.

Fase 8 — Manejo de errores

1. Definir un formato de error consistente, por ejemplo {error:{code,message,details}}.
2. Agregar handlers globales para 400, 401, 403, 404, 409, 422, 500 y 503.
3. Evitar exponer excepciones internas, SQL u otros detalles sensibles.
4. Diferenciar errores de validación, negocio, autenticación y disponibilidad.

Fase 9 — Comunicación entre microservicios

1. Mantener timeouts explícitos en llamadas HTTP.
2. Agregar reintentos controlados únicamente donde sean seguros.
3. Evitar reintentos automáticos en operaciones no idempotentes.
4. Propagar correlation/request IDs desde frontend → gateway → servicios.
5. Centralizar la configuración de URLs de los servicios.

Fase 10 — Observabilidad

1. Agregar logs estructurados.
2. No registrar contraseñas, JWT, API keys ni información sensible.
3. Agregar métricas básicas de requests, errores y tiempos de respuesta.
4. Separar endpoints /health y /readiness.
5. Registrar eventos importantes de negocio, como reserva de stock y creación de órdenes.

Fase 11 — Docker y despliegue

1. Agregar `.dockerignore` a cada servicio.
2. Ejecutar contenedores con usuario no root cuando sea posible.
3. Revisar y fijar versiones de imágenes base.
4. Fijar versiones de dependencias.
5. Usar multi-stage build para el frontend.
6. Evitar herramientas de desarrollo innecesarias en imágenes de producción.

Fase 12 — Frontend

1. Mejorar manejo de errores de API.
2. Agregar estados de loading, éxito, error y vacío.
3. Agregar validación de formularios.
4. Mejorar manejo de autenticación y expiración de sesión.
5. Agregar pruebas unitarias/componentes.
6. Agregar una prueba E2E del flujo registro → login → productos → carrito → checkout → historial.

Fase 13 — Documentación de API

1. Agregar OpenAPI/Swagger.
2. Publicar documentación navegable de endpoints.
3. Documentar autenticación JWT.
4. Documentar códigos de respuesta y ejemplos de requests/responses.
5. Documentar endpoints internos y dejar claro cuáles no deben exponerse públicamente.

Fase 14 — README y presentación del proyecto

1. Agregar captura del frontend.
2. Agregar diagrama de arquitectura.
3. Mostrar claramente el stack tecnológico.
4. Documentar requisitos, instalación y ejecución.
5. Documentar ejecución de tests y CI/CD.
6. Agregar enlace o sección de Swagger/OpenAPI.
7. Documentar decisiones de seguridad y arquitectura.
8. Documentar limitaciones conocidas.
9. Agregar roadmap de mejoras.

Fase 15 — GitHub y flujo profesional

1. Crear GitHub Actions.
2. Agregar plantilla de Pull Request.
3. Agregar plantillas de Issues.
4. Crear `CONTRIBUTING.md`.
5. Crear `SECURITY.md`.
6. Usar Issues para tareas técnicas.
7. Realizar cambios mediante Pull Requests cuando sea posible.
8. Mejorar mensajes de commit con Conventional Commits, por ejemplo feat:, fix:, test: y ci:.

Fase 16 — Architecture Decision Records (ADR)

1. Crear docs/adr/001-microservices.md.
2. Crear docs/adr/002-database-per-service.md.
3. Crear docs/adr/003-nginx-api-gateway.md.
4. Crear docs/adr/004-stock-reservation.md.
5. Explicar en cada ADR el contexto, decisión, alternativas y consecuencias.

Orden recomendado de implementación

1. Seguridad y eliminación de secretos.
2. PostgreSQL dentro de Docker Compose.
3. Migraciones con Flask-Migrate/Alembic.
4. Tests unitarios e integración.
5. GitHub Actions para CI.
6. Validación de APIs y manejo consistente de errores.
7. Idempotencia y consistencia de reserva de stock.
8. Correlation IDs, logs, health/readiness y métricas.
9. Mejoras del frontend y pruebas E2E.
10. Swagger/OpenAPI.
11. README, diagrama, capturas y ADRs.
12. Templates de GitHub, CONTRIBUTING, SECURITY y commits/PRs profesionales.

Las 5 mejoras con mayor impacto

- Eliminar todos los secretos y credenciales por defecto.
- Agregar tests unitarios e integración.
- Agregar GitHub Actions con lint, tests y build de Docker.
- Incluir PostgreSQL en Docker Compose para hacer el proyecto reproducible.
- Resolver o documentar correctamente la consistencia entre reserva de stock y creación de órdenes.

Objetivo final: convertir ShopMicro en un proyecto de portfolio que no solo demuestre que funciona, sino que también evidencie prácticas de desarrollo profesional: seguridad, pruebas, automatización, observabilidad, arquitectura consciente y documentación.